

1) Shallow copy and class Assignment

Given the following classes:

```
class Address;  
    bit [31:0] addr;  
    function new(bit [31:0] a);  
        addr = a;  
    endfunction  
endclass
```

```
class Packet;  
    Address addr_obj;  
    function new();  
        addr_obj = new(32'hDEADBEEF);  
    endfunction  
endclass
```

Task:

- a) Create two instances of Packet, pkt1 and pkt2.
- b) Assign pkt1 to pkt2.
- c) Modify the addr in pkt2 to 32'hCAFEBABE.
- d) Display the addr of both pkt1 and pkt2 to observe the effect of shallow copy.

SEP 17, 2025

NOW

Resources

Community

Help

Playgrounds

Pro

```

1 module shallow_copy_demo;
2   class Address;
3     bit [31:0] addr;
4     function new(bit [31:0] a);
5       addr = a;
6     endfunction
7   endclass
8   class Packet;
9     Address addr_obj;
10    function new();
11      addr_obj = new(32'hDEADBEEF); // default value
12    endfunction
13  endclass
14  initial begin
15    // a) Create two instances
16    Packet pkt1, pkt2;
17    pkt1 = new();
18    pkt2 = new();
19    $display("Before assignment:");
20    $display("pkt1 addr = %h", pkt1.addr_obj.addr);
21    $display("pkt2 addr = %h", pkt2.addr_obj.addr);
22    // b) Assign pkt1 to pkt2 (shallow copy)
23    pkt2 = pkt1;
24    // c) Modify pkt2 address
25    pkt2.addr_obj.addr = 32'hCAFEFEBABE;
26    // d) Display both
27    $display("After shallow copy modification:");
28    $display("pkt1 addr = %h", pkt1.addr_obj.addr);
29    $display("pkt2 addr = %h", pkt2.addr_obj.addr);
30  end
31 endmodule
32

```

2) Semaphore

You are tasked with designing a simulation where multiple processes attempt to access a shared resource. The resource can only be accessed by one process at a time. Using a semaphore, implement the following:

a) Create a semaphore with a single key.

b) Define two tasks: `access_resource` and `release_resource`.

c) `access_resource`: This task should attempt to acquire the semaphore. If successful, it should simulate accessing the shared resource for a certain duration (e.g., 10 time units) and then release the semaphore.

`release_resource`: This task should release the semaphore after accessing the resource.

d) Simulate multiple processes attempting to access the resource concurrently and observe the synchronization behavior.

```

1 // Code your design here
2 module semaphore_demo;
3     semaphore sem; // declare semaphore
4     // Task to access resource
5     task access_resource(string process_name);
6         sem.get(1); // try to acquire the semaphore
7         $display("%0t: %s got access to resource", $time,
process_name);
8         // Simulate resource usage
9         #10;
10        release_resource(process_name);
11    endtask
12    // Task to release resource
13    task release_resource(string process_name);
14        $display("%0t: %s releasing resource", $time, process_name);
15        sem.put(1); // release semaphore
16    endtask
17    initial begin
18        // a) Create semaphore with 1 key
19        sem = new(1);
20        // d) Multiple processes trying to access
21        fork
22            access_resource("Process_A");
23            access_resource("Process_B");
24            access_resource("Process_C");
25        join
26    end
27 endmodule
28
29

```

3) Mailbox

Design a producer-consumer system using a mailbox for interprocess communication. Implement the following:

- a) Create a mailbox with a specified capacity (e.g., 5 items).
- b) Define two tasks: producer and consumer.

producer: This task should generate data items (e.g., integers) and place them into the mailbox using the put method. If the mailbox is full, the producer should wait until space becomes available.

consumer: This task should retrieve data items from the mailbox using the get method and process them. If the mailbox is empty, the consumer should wait until data becomes available.

- c) Simulate the producer and consumer tasks running concurrently and observe the data exchange.

```

2 module mailbox_demo;
3     mailbox mbx; // declare mailbox
4     // Producer task
5     task producer;
6         int i;
7         for (i = 1; i <= 10; i++) begin
8             mbx.put(i); // put data
9             $display("%0t: Producer produced %0d", $time, i);
10            #5;
11        end
12    endtask
13    // Consumer task
14    task consumer;
15        int data;
16        forever begin
17            mbx.get(data); // get data
18            $display("%0t: Consumer consumed %0d", $time, data);
19            #10;
20        end
21    endtask
22    initial begin
23        // a) Create mailbox with capacity 5
24        mbx = new(5);
25        // c) Run producer and consumer concurrently
26        fork
27            producer();
28            consumer();
29        join_any
30        #100 $finish;
31    end
32 endmodule
33

```